

RAG Sistemi

Bileşen Bileşen

Ne inşa ettik, hangi kararı neden aldık, neyi ölçtük

github.com/Yigit033/rag-engineering-assistant

A · TEMEL

- 1 Domain modelleri + Protocol'ler
- 2 Merkezi config

B · VERİ HATTI

- 3 PDF loader
- 4 Token-bazlı chunker
- 5 Content-hash idempotency

C · ARAMA

- 6 bge-m3 embedder
- 7 FAISS IndexFlatIP store
- 8 Türkçe tokenizer + stemmer
- 9 Dense + BM25 retriever
- 10 RRF fusion
- 11 Cross-encoder rerank

D · CEVAP

- 12 Generation + halüsinasyon kontrolü

E · ÖLÇÜM

- 13 Evaluation harness

F · ÇATI

- 14 Kompozisyon kökü · API · Observability · CLI

EK

- 15 Bulunan sessiz hatalar · Deney günlüğü
-

Her bileşen aynı dört soruyu cevaplar:

- ① **Bu nedir** — kavram, tek satırlık tanımlar
- ② **Nasıl çalışır** — akış ve mekanizma
- ③ **Hangi karar neden** — alternatifi neden seçmedik
- ④ **Hangi hatayı imkânsız kılıyor** — tasarımın getirisi

Yeşil "KANIT" kutuları bu projede alınmış gerçek ölçümlerdir. Teorik iddia değil; sayılar kendi dokümanlarından ve kendi makinenden çıktı.

Turuncu kutular alternatifin neden yanlış olduğunu anlatır — kararın ağırlığı burada.

Domain Modelleri + Protocol'ler

domain/models.py · domain/protocols.py

① Bu nedir

- **Domain modelleri:** sistemin ortak dili. `Chunk`, `ScoredChunk`, `Answer`, `Citation`, `SourceRef`, `IngestReport`.
- **Protocol:** "bu şey **ne yapar**" sözleşmesi. "Nasıl yapar" yazmaz.
- Beş protokol: `Embedder` · `VectorStore` · `Retriever` · `Reranker` · `LLM`

② Nasıl çalışır

`domain/` klasörü **hiçbir üçüncü parti kütüphaneye bağlı değil**. Ne FAISS, ne sentence-transformers, ne FastAPI, ne pydantic.

Protocol **yapısal tipleme** sağlar: bir sınıfın protokolü sağlaması için ondan **miras almasına gerek yok** — aynı metot imzasına sahip olması yeterli.

Sonuç: **bağımlılık yönü tersine döner**. Kütüphane bize uyar, biz kütüphaneye değil.

```
Katman yönü:
api / cli / evaluation
    ↓ (yalnızca protokolleri görün)
ingestion / indexing / retrieval / generation
    ↓
domain ← en içte, hiçbir şeye bağlı değil
```

③ Hangi karar neden

Domain neden bu kadar katı korunuyor?

Domain en içteki katman; herkes ona bakar. Buraya bir kütüphane sızarsa o kütüphaneye **ömür boyu** bağlanırsın. LangChain'in `Document` sınıfını domain modeli yapan bir proje, LangChain'den bir daha çıkamaz — çünkü her katman o sınıfa bağımlı hale gelir.

Karar	Gerekçe
<code>frozen=True</code>	Model üretildikten sonra değiştirilemez. Pipeline'da sessiz mutasyon hatası imkânsız.
<code>slots=True</code>	Bellek tasarrufu + yanlış alan adına yazmayı engelleme.
<code>Chunk.id</code> içerikten türer <code>sha256(metin+dosya+sayfa)</code>	Aynı içerik → aynı kimlik, farklı süreçte bile. RRF birleştirmesinin anahtarı bu.
<code>embed_documents</code> ≠ <code>embed_query</code>	Retrieval asimetriktir. E5 tipi modeller <code>"query:"</code> / <code>"passage:"</code> öneki zorunlu kılar; unutursan hata almazsın , sadece kalite düşer. İki metodu ayırmak bu hatayı mimari olarak imkânsız kılar.
<code>Protocol</code> , <code>ABC</code> değil	ABC miras zorunlu kılar. Protocol kılmaz → üçüncü parti bir sınıfı hiç dokunmadan uyumlu sayabilirsiniz.
<code>DocumentStatus.NO_TEXT_LAYER</code> ayrı durum	"0 chunk üretildi" bir başarı değildir. Ayrı durum olmadan taranmış PDF sessizce kaybolur.
<code>ScoredChunk.retriever_hits</code>	Kaç retriever bu chunk'ı buldu? Hybrid aramanın gerçekten çalıştığını gösteren tek gözlemlenebilir sinyal.

④ Hangi hatayı imkânsız kılıyor

- Kütüphane kilidi — FAISS'ten Qdrant'a geçmek tek sınıf değiştirmek
- Sessiz mutasyon — `FrozenInstanceError` ile patlar
- Önek (prefix) unutma — çağırın taraf öneki hiç bilmez
- "0 chunk"ı başarı sayma
- Testte 2,3 GB model yüklemek zorunda kalmak

KANIT

```
143 test · gerçek model YÜKLENMEDEN · ~2 saniye
FakeEmbedder = 20 satır      FakeLLM = 12 satır      FakeStore = 25 satır

Chunk kimliği testi:
  id(a) != id(b)    True    ← farklı Python nesnesi
  a.id == b.id     True    ← AYNI kimlik → fusion çalışır
  a.id != c.id     True    ← farklı kaynak → farklı kimlik
```

Testler protokol arkasına sahte implementasyon geçiyor. Framework'e gömülü bir mimaride her test 2,3 GB model yüklerdi ve test yazmak pratikte terk edilirdi.

MÜLAKAT

1. Domain katmanını neden hiçbir kütüphaneye bağlamadın?
2. `Protocol` ile `ABC` farkı nedir, neden `Protocol`?
3. Sorgu ve dokümanı ayrı metotlarla vektörlemenin sebebi ne?

Merkezi Config

config.py

① Bu nedir

Tüm ayarlar **tek yerde**, tipli ve doğrulanmış. Kodda sihirli sayı yok.

Sekiz grup: `paths` · `embedding` · `chunking` · `retrieval` · `llm` · `generation` · `api` · `log`

Ortam değişkeniyle geçersiz kılma: `RAG_RETRIEVAL__TOP_K=3` (çift alt çizgi = iç içe alan).

② Nasıl çalışır

```
pydantic-settings BaseSettings
├─ iç içe BaseModel grupları (tip doğrulama)
├─ env / .env dosyası ile override
├─ lru_cache → uygulama boyunca TEK örnek
└─ model_validator → KATMANLAR ARASI kontrol
```

③ Hangi karar neden

Sihirli sayı neden yasak?

Bir RAG sisteminin kalitesi bir avuç sayıya bağlıdır: chunk boyutu, `fetch_k`, `top_k`, temperature. Bunlar kodun içine dağılmışsa:

- Neyi değiştirdiğini bilemezsin → **ölçüm anlamsızlaşır**
- Deney yapamazsın → iyileştirme tesadüfe kalır
- Ortam bazlı ayar yapamazsın → prod ile lokal ayrışır

Tek yerde toplandığında her ayar bir **deney değişkeni** olur.

Katmanlar arası kilitler — ikisi kritik

```
chunking.target_tokens > embedding.max_tokens → AÇILIŞTA PATLAR
sebebi: chunk'lar sessizce kesilir, o metin bir daha bulunamaz

retrieval.fetch_k < retrieval.top_k → AÇILIŞTA PATLAR
sebebi: recall kaybı geri alınamaz. Hiç aday olmayan bir chunk'ı
reranker kurtaramaz.
```

Karar	Gerekçe
<code>api_key_env</code> — sırrın adı , kendisi değil	Config nesnesi loglansa, hata izine girse veya diske yazılsa bile anahtar sızmaz.
<code>lru_cache</code> ile tek örnek	Ayarlar bir kez okunur. Testte <code>cache_clear()</code> ile sıfırlanır.
Ölçüm sonucu yorum olarak yazılı	<code>top_k = 3</code> satırının üstünde neden 3 olduğu ve hangi ölçümle seçildiği duruyor. Altı ay sonra "burası neden 3?" sorusu kendiliğinden cevaplanır.
Temperature varsayılanı 0.0	Bilgi çıkarmada zorunlu. Aynı soru farklı cevap verirse ne hata ayıklanabilir ne değerlendirilebilir.

④ Hangi hatayı imkânsız kılıyor

- **Eski projedeki en pahalı hata:** chunk 1000 **karakter**, model limiti 256 **token** → artık açılışta patlıyor
- `fetch_k < top_k` ile recall'ı geri alınamaz kaybetmek
- Sırların loga / hata izine sızması
- "Neyi değiştirdim?" belirsizliği

KANIT

```
chunk 2000 token vs embedding limiti 1024
→ ENGELLENDİ: "chunk'lar sessizce kesilir"
fetch_k=3 < top_k=10
→ ENGELLENDİ: "fetch_k >= top_k olmalı (geniş tara, sonra daralt)"

top_k deneyi TEK SATIRLA yapıldı:
rag-eval --k 3 → atıf doğruluğu %66,7 → %91,7
```

Ayar koda gömülü olsaydı bu deney kod değişikliği + yeniden kurulum isterdi. **Ölçüm disiplini, config disiplininin üzerine kurulur.**

MÜLAKAT

1. Ayarları tek yerde toplamanın ölçümle ne ilgisi var?
2. Chunk boyutu ile embedding limiti arasındaki ilişkiyi nasıl garanti ettin?
3. API anahtarını config'de neden tutmuyorsun?

PDF Loader

ingestion/loaders.py

① Bu nedir

Dosyayı okuyup **yapısı korunmuş metne** çevirir. Hattın girişi.

Üç iş: metni çıkar · cerrahi temizle · dosya parmak izini üret.

② Nasıl çalışır

PDF → pypdf → sayfa sayfa metin → normalize → LoadedDocument + content_hash

Normalizasyon — 4 hedefli işlem

İşlem	Neden
Unicode NFC	"ğ" tek kod noktası mı, "g + birleşen aksan" mı? İkisi farklı bayt dizisi → aynı kelime hiç eşleşmez.
Satır sonu tirişi birleştir	daya-\nnım → dayanım
Satır içi fazla boşluk tekler	[^\S\n] deseni: \n KORUNUR
3+ boş satır → 2	Paragraf sınırı için 2 yeter

Satır ve paragraf yapısı olduğu gibi kalır. Bu dosyanın en kritik kararı.

③ Hangi karar neden

Neden "temizlemek" değil, "yapıyı bozmadan temizlemek"?

Belgelerde anlamın büyük kısmı **düzenden** gelir: paragraf sınırı, başlık satırı, Etiket: değer hizalaması. Naif bir \s+ deseni tüm boşlukları tekler — satır sonları dahil. Metin "temiz" görünür ama etiket-değer ilişkisi yok olur ve sonraki her aşama çöker. Temizlik ile yıkım arasındaki fark **tek karakterlik** bir desen detaydır.

Karar	Gerekçe
Tire birleştirmede sadece harfler [^\W\d_]	\w rakamı da yakalar → 2020-\n2021 = 20202021 . Tarih aralığı sessizce bozulur.
Sayfa başına hata yalıtımı	Tek bozuk sayfa tüm dokümanı düşürmez. Bozuk PDF yaygın; kısmi başarı > tam başarısızlık. Ama hata yutulmaz — loglanır ve sayılır.
Hash akıllı (1 MB blok)	Tüm dosyayı RAM'e almaz. Büyük PDF'te fark eder.
pypdf tek yerde bilinir	PyMuPDF / docling'e geçiş = sadece bu dosya. Üst katmanlar hangi kütüphaneyi kullandığımızı bilmez.
pypdf logger'ı bastırılır	Aynı bilgiyi biz yapılandırılmış olay olarak raporluyoruz. Kütüphanenin ham gürültüsü anlamlı çıktıyı boğuyordu.
LoaderRegistry	Yeni format = yeni loader yazmak, mevcut kodu değiştirmemek (açık/kapalı ilkesi). OCR loader'ı ileride buraya eklenir.

④ Hangi hatayı imkânsız kılıyor

- Yapı yok etme → chunking ve extraction'ın çökmesi
- Aynı Türkçe kelimenin iki farklı bayt dizisi olması
- Tarih/sayı aralığının bozulması
- Bir bozuk sayfanın 200 sayfalık dokümanı çöpe atması
- Taranmış PDF'in sessizce kaybolması
- Kütüphaneye ömür boyu bağlanmak

KANIT — BOZUK PDF ÜZERİNDE

```
SmartHull: pypdf "incorrect startxref pointer" + 12 decompression error
  → loader ÇÖKMEDİ. 7 sayfa okundu, 0 karakter çıktı.
  → has_text_layer=False → durum: no_text_layer (OCR gerekli)

SmartSafe: 5 sayfa, 4932 karakter → durum: ok
```

no_text_layer ayrı bir durum. "0 chunk" **asla başarı sayılmaz** — yoksa o doküman sisteme hiç girmez ve kimse fark etmez.

MÜLAKAT

1. PDF'ten metin çıkarırken neyi korumak zorundasın, neden?
2. Bir sayfa okunamazsa ne yaparsın?
3. Aynı kütüphaneye bağlı kalmamak için ne yaptın?

Token-Bazlı Chunker

ingestion/chunker.py

① Bu nedir

Dokümanı **aranabilir parçalara** böler. Üç sebeple zorunlu:

- Embedding modelinin token limiti var
- LLM'e tüm dokümanı veremezsin
- 500 sayfa tek vektöre sıkışırsa anlam erir

② Nasıl çalışır — iki sert garanti

1. Hiçbir chunk model limitini AŞMAZ (garanti, hedef değil)
2. Bölme noktaları YAPIYI takip eder:
paragraf → cümle → (mecbursa) kelime

Algoritma

```
sayfa → paragraflara ayır
  → paragraf bütçeye sığıyorsa OLDUĞU GİBİ birim yap ← en iyi durum
  → sığmıyorsa cümlelere ayır (Türkçe kısaltmaları koruyarak)
  → tek cümle bile sığmıyorsa kelime bazında sert kes + UYARI LOGLA
  → birimleri bütçe dolana kadar grupla
  → gruplar arasında örtüşme bırak
```

③ Hangi karar neden

Neden karakter değil token?

Embedding modelinin limiti **token** cinsindendir. Karakterle ölçmek farklı birimleri karşılaştırmaktır. Türkçe'de karakter/token $\approx 2,5$ (İngilizce'de ≈ 6) — yani aynı karakter sayısı Türkçe'de **2,4 kat** daha fazla token demek. Sonuç: chunk sessizce kesilir, kesilen metin bir daha asla bulunamaz, hata mesajı da alınmaz.

Karar	Gerekçe
Chunk sayfa sınırını aşmaz	Atıf için. "rapor.pdf s.7" diyebilmek için chunk'ın hangi sayfadan geldiği kesin olmalı. İki sayfayı birleştiren chunk'a doğru sayfa numarası verilemez.
Sert limit AssertionError ile korunur	Limit aşımı sessizce geçemez; programlama hatası olarak patlar.
Token sayımı birleştirilmiş metin üzerinden	Birimleri tek tek toplamak ayırcının (<code>\n\n</code>) maliyetini saymaz ve toplamı olduğundan az gösterir.
<code>count_tokens</code> enjekte edilir	Chunker embedder'ın tamamına değil sadece token sayma yeteneğine ihtiyaç duyar (en dar arayüz). Testte <code>len(t.split())</code> geçmek yeterli.
Sert kesme loglanır	Buraya düşmek uyarı işaretidir (tablo dökümü, satır sonu olmayan liste). Sessiz kesme kabul edilemez.

Örtüşme (overlap) — düzeltilen tasarım hatası

İlk tasarımda örtüşme **paragraf düzeyinde** taşınyordu. Paragraf örtüşme bütçesinden büyükse örtüşme **hiç uygulanmıyordu** — ve gerçek ayarlarda (hedef 400, örtüşme 80) paragrafların çoğu bütçeden büyük. Yani örtüşme fiilen **kapalıydı**.

Düzeltilme: örtüşme artık metin **soneki** olarak alınıyor — önce cümle sınırından, gerekirse kelime bazında. Bütçe her iki durumda kesin.

Örtüşme neden var: bir bilgi tam chunk sınırına düşerse (öznesi bir chunk'ta, yüklemi diğerinde) hiçbir chunk o bilgiyi tek başına cevaplayamaz.

④ Hangi hatayı imkânsız kılıyor

- Chunk'ın model limitini aşp **sessizce kesilmesi**
- Cümlelerin ortadan bölünüp iki chunk'ın da faydasız olması
- Örtüşmenin sessizce devre dışı kalması
- İki sayfaya yayılan chunk'a yanlış sayfa numarası verilmesi

KANIT

```
SmartSafe (5 sayfa):  
6 chunk · token min/ort/max: 97 / 235 / 352 · limit 1024  
limiti aşan chunk: 0  
  
Eski sistemle kıyas:  
8 chunk, 4'ü 256 token limitini aşıyordu → 420 token SESSİZCE siliniyordu
```

MÜLAKAT

1. Chunk boyutunu neye göre seçersin? Neden token, neden karakter değil?
2. Overlap neden var, nasıl ölçülür?
3. Chunk'ın sayfa sınırını aşmaması neden önemli?

Content-Hash Idempotency

ingestion/manifest.py · ingestion/pipeline.py

① Bu nedir

- **Idempotency:** aynı işlemi 2 kez yapmak, 1 kez yapmakla aynı sonucu vermeli.
- **Content hash:** içerikten üretilen parmak izi (SHA-256).
- **Manifest:** hangi dosya, hangi içerikle, ne zaman işlendi — diskteki defter.

Cevapladığı soru: "Bu dosyayı yeniden işlemem gerekiyor mu?"

② Nasıl çalışır — İKİ AYRI hash var

Bu ayrım işin kalbi. Aynı teknik, **iki farklı soru**:

	Dosya hash'i	Chunk hash'i
Neyden	Dosyanın baytları	metin + dosya + sayfa
Hangi soru	"Bu doküman değişti mi?"	"Bu metin parçası aynı mı?"
Nerede	Manifest — ingestion atlama	Index — tekilleştirme + RRF anahtarı

Atlama kararı — BEŞ koşulun HEPSİ sağlanmalı

1. Manifest'te kaydı var
2. İçerik hash'i aynı → dosya değişmemiş
3. Durumu OK → NO_TEXT_LAYER / FAILED tekrar denenir
4. Embedding modeli aynı → model değiştiyse index geçersiz
5. Chunk'ları GERÇEKTEN index'te → manifest ile index tutarlı

③ Hangi karar neden

Neden değiştirilme zamanı (mtime) değil, içerik hash'i?

mtime; kopyalama, git checkout, yedekten geri yükleme, dosya senkronizasyonu gibi işlemlerde **içerik hiç değişmeden** değişir → sistem dosyayı "değiştirilmiş" sayıp baştan işler (boşa CPU). Tersine de mümkündür: farklı içerik aynı mtime'la gelirse hiç işlenmez. İçerik hash'i tek doğru ölçüttür.

5. koşul neden kritik?

Manifest "işlendi" derken index boş olabilir (silinmiş, taşınmış, bozulmuş). İki kaynağı karşılaştırmadan "işlenmiş" demek, sistemin **boş bir index'le** "her şey hazır" demesidir.

Karar	Gerekçe
Manifest yazımı atomik (geçici dosya → <code>replace</code>)	Yazma sırasında süreç ölürse (Ctrl+C) yarım JSON kalır ve manifest bir daha okunamaz — sistem tüm geçmişini kaybeder.
Bozuk manifest sistemi durdurmaz	En kötü senaryo: her şey yeniden işlenir (yavaş ama doğru). Sessizce yutulmaz, loglanır.
Güncellenen dosyanın eski chunk'ları silinir (<code>remove_by_file</code>)	Yapılmazsa index'te artık hiçbir dosyaya ait olmayan "hayalet" chunk'lar kalır ve aramada eski bilgi döner.
<code>--force</code> seçeneği	Chunk ayarları veya normalizasyon mantığı değiştiğinde gerekir: bunlar dosya içeriğine yansımadağı için hash aynı kalır.

④ Hangi hatayı imkânsız kılıyor

- Her çalıştırmada tüm embedding'lerin yeniden hesaplanması
- Aynı chunk'ın index'e iki kez girmesi
- Kopyalama/checkout sonrası gereksiz yeniden işleme
- Boş index'le "her şey hazır" demek
- Güncellenen dosyanın hayalet chunk'ları
- Yarım yazılmış manifest ile tüm geçmişi kaybetmek

KANIT

```
1. çalışma: 5.14 s · 6 chunk eklendi
2. çalışma: 0.25 s · 0 chunk eklendi ← SmartSafe: skipped
                                   SmartHull: yeniden denendi
                                   (0 chunk asla "tamamlandı" değil)
```

20 kat hızlanma ve sıfır yan etki. Taranmış PDF her seferinde tekrar deneniyor — çünkü aradaki zamanda OCR desteği eklenmiş olabilir.

MÜLAKAT

- Idempotency'yi neye göre kurdun, neden mtime değil?
- Manifest "işlendi" diyor ama index boş — nasıl yakalarsın?
- Chunk ayarını değiştirdin, hash aynı. Ne yaparsın?

bge-m3 Embedder

indexing/embedder.py

① Bu nedir

Metni **vektöre** çevirir. Yakın anlam → yakın vektör. Arama böylece "en yakın komşuyu bul" problemine dönüşür.

BAAI/bge-m3 : çok dilli (100+ dil), 1024 boyut, 8192 token bağlam.

② Nasıl çalışır

Doküman → `embed_documents()` → (n, 1024) matris, L2-normalize
Soru → `embed_query()` → (1024,) vektör, L2-normalize

Model pipeline'ı: Transformer → Pooling (mean) → Normalize

③ Hangi karar neden

Neden MiniLM değil? (ölçülmüş gerekçe)

Eski sistem `all-MiniLM-L6-v2` kullanıyordu — salt İngilizce eğitilmiş. Tokenizer'ı Türkçe diyakritikleri (ç, ş, ğ, ı, ö, ü) siliyor. Sonuç aşağıdaki kanıt kutusunda: sigortadaki "poliçe" ile İngilizce "police" **birebir aynı vektör** oluyordu.

Karar	Gerekçe
Tembel yükleme (<code>cached_property</code>)	Model 2,3 GB. <code>__init__</code> 'te yüklenirse her import bellek doldurur, testler yavaşlar, CLI açılışı saniyeler sürer.
Çıktı L2-normalize (sözleşme)	Store tarafında inner product doğrudan cosine'a eşit olur; "normalize etmeyi unutma" hatası ortadan kalkar.
<code>max_tokens</code> modelin limitinin altında	bge-m3 8192 kabul eder, biz 1024 kullanıyoruz. Çok uzun chunk arama hassasiyetini düşürür. Modelin limitini aşmak ise sessiz kesme demektir → ikisinin küçüğü alınır.
Önce yerel ön bellek , sonra Hub	Model ön bellekte olmasına rağmen her açılışta HuggingFace Hub'a soruluyordu. Hub yavaşladığında tamamen yerel bir servis düştü (fiilen yaşandı).
GPU LLM'e bırakıldı, embedder CPU'da	8 GB VRAM'de embedder + reranker + LLM sıkışır. Korpus küçük olduğu için CPU darboğaz değil.

KANIT — AYNI ÜÇ ÖLÇÜM, İKİ MODEL

Ölçüm	MiniLM (eski)	bge-m3 (yeni)
<code>cos("poliçe", "police")</code>	1.0000 (aynı vektör!)	0.6816 (ilişkili ama ayrı)
<code>cos("gemi gövdesi dayanımı", "hull strength")</code>	0.029	0.6455 · 22 kat
<code>cos("gemi gövdesi dayanımı", "yemek tarifi")</code>	0.358	0.375

Kritik olan sıralamanın düzelmesi: eskiden çeviri (0.029), alakasız metinden (0.358) **daha uzaktı** — anlamsal uzay ters çalışıyordu. Şimdi çeviri 0.645, alakasız 0.375. Doğru yönde.

MÜLAKAT

1. Türkçe için model seçerken hangi kontrolleri yaparsın?
2. Embedding modelini değiştirdin — index'e ne yapman gerekir, neden?
3. Çıktının normalize olmasını neden sözleşmeye bağladın?

FAISS IndexFlatIP Store

indexing/store.py

① Bu nedir

Vektörleri saklar ve "en yakın k" sorgusunu cevaplar. Aynı zamanda chunk'ların **tek doğruluk kaynağı**.

FAISS bir veritabanı değil, arama kütüphanesi. Kalıcılık/filtreleme/çok kullanıcı yok — onları biz kuruyoruz.

② Nasıl çalışır

```
bellek: IndexFlatIP (vektörler) + list[Chunk] + {chunk_id → satır}
diskte: index.faiss + chunks.json + meta.json
(satır sırası FAISS sırasıyla BİREBİR aynı olmak zorunda)
```

③ Hangi karar neden

Karar	Gerekçe
IndexFlatIP, IndexFlatL2 değil	Vektörler normalize geldiği için inner product doğrudan cosine'a eşit ($\ a\ =\ b\ =1 \Rightarrow a \cdot b = \cos$). Skor okunabilir: 1.0 = aynı, 0 = ilgisiz. $1/(1+d)$ gibi uydurma dönüşüme gerek kalmaz.
Flat = kaba kuvvet	%100 kesin. ~100K vektöre kadar doğru tercih; bu ölçekte ANN (IVF/HNSW) gereksiz karmaşıklık.
Kalıcılık JSON , pickle değil	Pickle açmak kod çalıştırmaktır . Index başka makineden/yedekten/iş arkadaşından geliyorsa güvenlik açığıdır. JSON: güvenli, okunabilir, diff'lenebilir, dille bağımsız.
Index'e model kimliği yazılır	Model değişirse eski vektörler geçersizdir. Bu kontrol olmadan sistem hata vermeden çöp sonuç döndürür.
Chunk'ların tek kaynağı store	BM25 de aynı chunk kümesine ihtiyaç duyar. İki ayrı yerden iki kopya yüklenirse kimliğe dayalı birleştirme sessizce bozulur.
Kaydetme atomik	Yarım index + tam chunk listesi = tutarsız durum. Tutarsız index, hatalı sonuçtan daha kötüdür çünkü fark edilmez.
remove_by_file index'i yeniden kurar	IndexFlat seçmeli silmeyi verimli desteklemiyor. Bu ölçekte maliyet önemsiz; karşılığında hayalet chunk sorunu tamamen kalkıyor.

LangChain'in FAISS sarmalayıcısı neden allow_dangerous_deserialization=True ister?

Çünkü pickle kullanıyor. O bayrağı geçmek zorunda kalmanın kendisi bir tasarım kokusudur. JSON'a geçince bayrak da, risk de ortadan kalkıyor.

④ Hangi hatayı imkânsız kılıyor

- Pickle üzerinden kod çalıştırma riski
- Model/index uyumsuzluğunun sessizce çöp sonuç üretmesi
- Boyut uyumsuzluğu

- Tutarsız (yarım) index
- Hayalet chunk'lar
- Uydurma skor dönüşümleri

KANIT

Diskte: chunks.json · index.faiss · meta.json (pickle YOK)

```
meta: {"version":1, "dimension":1024,  
      "embedder_model_id":"BAAI/bge-m3",  
      "count":6, "index_type":"IndexFlatIP"}
```

Kilit testleri:

model 'baska/model' ile aç → ENGELLENDİ: "Vektör uzayları uyumsuz"

boyut 384 ile aç → ENGELLENDİ: "Index boyutu 1024, model 384"

MÜLAKAT

1. Normalize vektörlerde inner product neden cosine'a eşittir?
2. Flat yerine ne zaman IVFFlat / HNSW?
3. Index'i pickle ile saklamanın riski nedir?

Türkçe Tokenizer + Stemmer

text.py · retrieval/tokenize.py

① Bu nedir

BM25 (kelime araması) için metni **token'lara** ayırır. Dense arama modelin kendi tokenizer'ını kullanır; sparse tarafta tokenizasyonu **biz** yaparız.

Yanlış yapılırsa BM25 sessizce zayıflar — hata vermez, sadece daha az eşleşme bulur ve hybrid aramanın yarısı boşa çalışır.

② Nasıl çalışır

metin → NFC normalize → Türkçe-güvenli küçük harf → alfanümerik token'lar
→ yinelemeli son ek kırma → gövde sonu sertleştirme

③ Hangi karar neden — Türkçe'nin üç tuzağı

Tuzak 1: "İ".lower() bozuk sonuç verir

Python'da "İ".lower() → 'i' yani **i + BİRLEŞEN NOKTA** = iki karakter. Naif .lower() / .casefold() kullanan her Türkçe kod burada sessizce bozulur. Ayrıca "I".lower() → 'i' verir; doğrusu 'ı'.

Çözüm: Türkçe büyük harfleri önce elle eşle, sonra lower() çağır. Bu mantık text.py 'de **tek yerde** durur.

Tuzak 2: tek geçişli kırma asimetrik gövde üretir

Türkçe'de sesliyle biten gövdeye ek gelirken **kaynaştırma harfi** girer: "poliçe" + "in" → "poliçe-n-in". Tek geçişle kırınca:

```
"poliçenin" → "nin" atılır → "poliçe"
"poliçe" → "e" atılır → "poliç" ← EŞLEŞMİYOR
```

Çözüm: sabit noktaya kadar yinele. İkisi de "poliç"e iner. Dilbilimsel doğruluk değil, **tutarlılık** önemli — sorgu ile doküman aynı gövdeye insin.

Tuzak 3: ünsüz yumuşaması

"güvenlik" + "in" → "güvenliğin". Ek atıldıktan sonra geriye yumuşak biçim kalır ve eksiz halle eşleşmez. **Çözüm:** gövde sonunu sertleştir (ğ→k, b→p, c→ç, d→t), **koşulsuz** — yoksa yeni bir asimetri doğar.

Karar	Gerekçe
Sonek listesi kodla sıralanır	"En uzun eşleşme kazanır" kuralı elle sıralamaya bırakılırsa biri araya kısa ek ekleyince sessizce bozulur. Ölüldü: "i" ekiyle "gövdesi" → "gövdes" kalıyordu.
<code>_MIN_STEM_LENGTH = 4</code>	Aşırı kırpmayı sınırlar. Bedeli: 4 harften kısa gövdeler kırılmaz (kod / kodun eşleşmez) — bilinçli ödünleşim.
Katlama tek modülde (text.py)	Aynı hata iki katmanda çıktı: tokenize doğru yapıyordu, answerer casefold() kullanıp bozuluyordu. Kesişen mesele → ortak yer.

④ Hangi hatayı imkânsız kılıyor

- Büyük harfli Türkçe metnin eşleşmemesi
- Ekli/eksiz biçimlerin buluşmaması
- Ünsüz yumuşamasının kaçırılması
- Aynı mantığın iki kopyasından birini düzeltip diğerini unutmak

KANIT

```
"İ".lower() → 'i' (İKİ karakter – bozuk)
normalize("İSTANBUL") → 'istanbul'      normalize("IŞIK") → 'ışık'
```

Ekli/eksiz eşleşme: 10/11

poliçe/poliçenin ✓ gemi/gemiler ✓ teminat/teminatı ✓
gövde/gövdesi ✓ tarih/tarihinde ✓ güvenlik/güvenliğin ✓
kitap/kitabın ✓ kod/kodun X (4 harf sınırı – bilinçli)

Aşırı kırpma yok: kalem#kale · gemi#gemlik · rapor#rapel · bina#binlik

MÜLAKAT

1. Türkçe'de `.lower()` neden güvenli değil?
2. Stemmer'ın amacı ne? Tek geçiş neden yetmez?
3. Aşırı kırpma ile kaçırma arasında nasıl denge kurdun?

Dense + BM25 Retriever

retrieval/retrievers.py

① Bu nedir

- **Dense (embedding):** anlam benzerliğine bakar.
- **Sparse (BM25):** kelime örtüşmesine bakar.
- İkisi de aynı `Retriever` protokolünü sağlar → birbirinin yerine geçebilir.

② Nasıl çalışır — neden İKİSİ birden

	Dense	Sparse (BM25)
Güçlü	Eşanlam, parafraz ("araç" ↔ "otomobil")	Özel isim, kod, seri no ("Intenseye", "VF7OBBHY...")
Zayıf	Nadir token'ları vektörde "eritir"	Eşanlamı hiç göremez

Hata modları FARKLIDIR ve birbirini tamamlar. Biri tamamen bozulsaydı diğeri sistemi ayakta tutar — bu ölçülmüş bir gerçek, teorik iddia değil.

③ Hangi karar neden

BM25 elemesi neden skora değil, token örtüşmesine bakıyor?

BM25'in IDF terimi $\log((N-n+0.5)/(n+0.5))$ biçimindedir ve $n/N \approx 0.5$ olduğunda **sıfır** olur. Küçük korpustalarda bu sürekli olur: $N=2$ doküman, terim 1'inde geçiyorsa $idf = \log(1) = 0 \rightarrow$ skor 0.

Yani **gerçek bir eşleşme 0 puan alır**. $score > 0$ ile elemek bu eşleşmeleri sessizce atar ve sparse retriever küçük korpusta fiilen çalışmaz.

Doğru soru: "skoru pozitif mi?" değil, "sorgu kelimesi bu chunk'ta geçiyor mu?" Sıralama yine BM25'e göre yapılır.

Karar	Gerekçe
BM25 chunk'ları store'dan alır	Dense sonuçlarıyla aynı kaynak. İki ayrı kopya yüklenirse kimliğe dayalı birleştirme bozulur. (Kimlik içerikten türediği için ikinci güvenlik katmanı da var.)
BM25 indeksi chunk sayısı değişince yeniden kurulur	Yoksa ingestion sonrası yeni chunk'lar sparse aramada hiç görünmez — sessiz kayıp.
Dense skor döndürmüyor , sadece sıra	RRF skora değil sıraya bakar; gereksiz normalizasyon karmaşıklığı kaldırıldı.

④ Hangi hatayı imkânsız kılıyor

- Küçük korpusta sparse retriever'in sessizce devre dışı kalması
- Yeni eklenen chunk'ların sparse aramada görünmemesi
- İki farklı chunk kopyasıyla çalışmak

Aynı sorguda skorlar:

dense (cosine): 0.61 / 0.55 / 0.50 ← [0,1] aralığı
BM25 : 2.81 / 0.76 ← sınırsız

Bu ikisini TOPLAMAK anlamsız → RRF'in var olma sebebi.

Eleme düzeltmesi (skor → token örtüşmesi):

sparse sonuç sayısı: 2 → 4
hybrid uzlaşma : 2/3 → 3/3

MÜLAKAT

1. Hybrid arama neden tek başına dense'ten iyi?
2. Hangi sorgu tipinde BM25 kazanır, neden?
3. BM25 skoru 0 olan bir sonuç ne demek?

RRF Fusion

retrieval/fusion.py · retrieval/pipeline.py

① Bu nedir

Reciprocal Rank Fusion: birden çok retriever'in sonuçlarını, **skorlarına değil sıralarına** bakarak birleştirir.

$$\text{RRF}(d) = \sum \frac{1}{(k + \text{rank}(d))} \quad k = 60$$

② Nasıl çalışır — `k=60` ne işe yarar

`k=0` olsaydı 1. sıra 1.0, 2. sıra 0.5 puan alırdı → tek retriever'in birincisi her şeyi ezerdi. `k=60` ile 1. sıra 0.01639, 2. sıra 0.01613 → aralar daralır.

Sonuç: **iki listede birden geçen doküman, tek listede birinci olanı geçer.** RRF'in bütün değeri bu "uzlaşma ödülü"nde.

Örnek (`k=60`):
 A: dense #1, sparse yok → $1/61 = 0.01639$
 B: dense #3, sparse #2 → $1/63 + 1/62 = 0.03200 \leftarrow \text{KAZANIR}$

B hiçbir listede birinci değil, ama iki farklı yöntem bağımsız olarak onu işaret etmiş.

③ Hangi karar neden

Neden skor değil sıra?

BM25 skorları sınırsız pozitif ($0 \dots \infty$), cosine $[-1, 1]$. Doğrudan toplamak anlamsız; normalize etmek gerekir, normalizasyon ise veri dağılımına duyarlıdır ve her veri setinde yeniden ayar ister. **Sıra kullanmak ölçek problemini kökten kaldırır ve ayar gerektirmez.**

Birleştirme anahtarı: `chunk.id` — nesne kimliği ASLA

Eski sistemde anahtar olarak Python'un `id()` fonksiyonu kullanılıyordu — yani **bellek adresi**. Aynı içerik farklı nesnelerde tutulunca hiçbir eşleşme bulunmaz, skorlar toplanmaz ve fusion fiilen **"iki listeyi arka arkaya ekleme"**ye dönüşür. Hata mesajı da alınmaz.

Üstüne: vektör store'u iki kez yükleniyordu → pickle'dan yeni nesneler → kimlikler zaten hiç eşleşemezdi. Yani **fusion hiç çalışmamıştı.**

Geniş tara, sonra daralt

`fetch_k = 20` → RECALL (doğru cevabı adaylar arasına al)
`top_k = 3` → PRECISION (LLM'e az ve isabetli chunk ver)

Recall kaybı geri alınamaz: hiç aday olmayan bir chunk'ı reranker kurtaramaz.

④ Hangi hatayı imkânsız kılıyor

- Farklı ölçekli skorları toplamak
- Her veri setinde ağırlık ayarlamak zorunda kalmak

- Fusion'ın sessizce çalışmaması (içerik tabanlı anahtar)
- Doğrudan top_k çekip recall'ı kaybetmek

KANIT — FUSİON GERÇEKTEN ÇALIŞIYOR MU?

Ölçülebilir sinyal: "kaç sonuç İKİ retriever'da birden geçti?"

eski sistem (id() anahtarı): matematiksel olarak HER ZAMAN 0
yeni sistem: 3/3

Birim test (aynı içerik, ayrı nesneler):

id(first) != id(second) → fused uzunluğu 1, retriever_hits = 2

Bu sayı sürekli 0 ise birleştirme bozuk demektir. retriever_hits alanı bu yüzden rerank sonrasında da **korunuyor** — kaybedilirse sinyal yok olur.

MÜLAKAT

1. RRF'te k sabiti ne işe yarar? k=0 olsa ne olur?
2. Skor fusion ile rank fusion arasında neden rank'ı seçtin?
3. fetch_k neden top_k 'dan büyük?

Cross-Encoder Rerank

retrieval/rerank.py · resources.py

① Bu nedir

Adayları **daha isabetli** bir modelle yeniden sıralar. Pipeline'ın precision aşaması.

② Nasıl çalışır — bi-encoder vs cross-encoder

	Bi-encoder (retriever)	Cross-encoder (reranker)
Nasıl	Sorgu ve doküman AYRI vektörlenir	Sorgu + doküman TEK girdi olarak modele verilir
Hız	Doküman vektörleri önceden hazır → milisaniye	Her çift için model çalışır → yavaş
Kalite	Etkileşimi kaçıır	İnce ilişkiyi doğrudan modeller
Ölçek	Milyonlarca doküman	Onlarca aday

Bu yüzden mimari iki aşamalı: **geniş tara (ucuz) → daralt (pahalı ama isabetli)**.

③ Hangi karar neden

Ölçülmüş çökme: isteğe bağlı bileşen tüm servisi düşürdü

API sunucusu, embedder (2,3 GB) yüklükten reranker'ı (2,2 GB) yüklerken **işletim sistemi tarafından öldürüldü**. Dikkat: **Python istisnası fırlatılmadı** — yani `try/except` bunu yakalayamaz. Süreç sessizce gitti.

Bu, "önce dene, hata olursa yakala" yaklaşımının işlemediği bir durumdur. Tek doğru yöntem **denemeden önce kontrol** etmektir.

Karar	Gerekçe
Bellek ön kontrolü (<code>resources.py</code>)	Yüklemeden önce boş RAM ölçülür. Güvenlik payı 1,4× — dosya boyutu kadar bellek asla yetmez (float32 açılım + ara tensörler).
Zarif düşüş (graceful degradation)	Yer yoksa reranker devre dışı kalır, adaylar RRF sırasıyla geçirilir. Kaliteyi artıran bir bileşen, zorunlu işlevi düşürmemeli.
Durum <code>/ready</code> 'de raporlanır	Sessizce kaybolmuş bir kalite katmanı, en zor fark edilen gerilemedir.
Reranker hazırlığı etkilemez	İsteğe bağlı olduğu için <code>ready=true</code> kalır. Zorunlu saymak, çalışabilir bir servisi gereksizce kapatmak olurdu.
Tembel yükleme	2,2 GB. Kullanılmazsa belleğe hiç alınmaz.
<code>retriever_hits</code> korunur	Fusion'daki uzlaşma bilgisi kaybolmasın.
<code>min_score</code> eşiği (opsiyonel)	Eşiğin altındaki aday atılır → "hiç alakalı sonuç yok" durumu LLM'e boş bağlam olarak iletilir → model çekimser kalır. Uydurmaktan iyidir.

④ Hangi hatayı imkânsız kılıyor

- İsteğe bağlı bir bileşenin tüm servisi düşürmesi
- `try/except` 'in yakalayamadığı OS-kill

- Kalite katmanının sessizce kaybolması

KANIT

Bellek kontrolü:

RAM 14.7 GB toplam • 0.65 GB boş

reranker (2.3 GB × 1.4 = 3.22 GB gerekli) → yer var mı: False

→ reranker.disabled "sistem çalışmaya devam eder;
yalnızca sıralama hassasiyeti düşer"

Reranker aktifken (RAM boşken) sıralama etkisi:

RRF skorları: 0.0328 / 0.0323 / 0.0159 ← birbirine yakın

Rerank sonrası: 0.6595 / 0.0283 / 0.0012 ← keskin ayrım

3. sıra s.2'den s.3'e değişti

MÜLAKAT

1. Bi-encoder ve cross-encoder'ı pipeline'da nereye koyarsın, neden?
2. Bellek yetmezliğini try/except ile neden yakalayamazsın?
3. İsteğe bağlı bir bileşen düşerse sistem ne yapmalı?

Generation + Halüsinasyon Kontrolü

generation/ (prompt · llm · answerer · factory · openai_compat)

① Bu nedir

Getirilen bağlamı LLM'e verip **denetlenabilir** cevap üretir: kaynak gösteren, ya da bilgi yoksa **çekimser kalan**.

② Nasıl çalışır — beş savunma katmanı

1. BOŞ BAĞLAMDA MODELİ HİÇ ÇAĞIRMA
Aday yoksa LLM'e boş bağlam göndermek uydurma davetiyesidir.
2. PROMPT'TA AÇIK ÇEKİMSER KALMA İZNİ
"Bilmiyorsan şunu yaz" demezsen model boşluğu uydurmayla doldurur.
3. NUMARALI BAĞLAM + ZORUNLU ATIF
Her chunk [1],[2] diye numaralanır; iddialar bunlara bağlanır.
4. ATIF DOĞRULAMA
Var olmayan numaraya atıf ayıklanır ve loglanır.
"Kaynak göstermiş gibi görünen" cevap en sinsi halüsinasyondur.
5. GROUNDEDNESS DENETİMİ (opsiyonel, ikinci LLM çağırısı)
Cevaptaki iddiaların kaç bağlamda destekleniyor?

③ Hangi karar neden

Model çıktısı SESSİZCE değiştirilmez

Atıfsız bir cevap geldiğinde onu silip kendi metnimizi koymuyoruz. Sonucu **işaretliyoruz** (`is_grounded`) ve logluyoruz. Sessiz düzeltme sorunu gizler ve ölçümü bozar. **Görünür başarısızlık, gizli "düzeltme"den her zaman iyidir.**

Karar	Gerekçe
Prompt koda gömülü değil , versiyonlu dosya	Prompt bir model parametresidir . Sürümlenmezse hangi prompt hangi skoru ürettiğini bilemezsin, v1 ile v2'yi karşılaştıramazsın, üretimde hangi talimatın verildiğini kanıtlayamazsın.
Eksik prompt değişkeni KeyError	Yarısı boş bir prompt'la modeli çağırmak sessizce bozuk sistemdir.
Reasoning izi ayıklama (<code><think></code>)	Düşünme bloğunun içinde de "[1]" geçer. Ayıklanmazsa atıf çözücü onu gerçek atıf sayar → dayanağı olmayan cevap "kaynaklı" görünür. Akışta da bastırılır (etiket iki parçaya bölünebilir → durum tutulur).
Çekimserlikte atıflar temizlenir	Bazı modeller "bu bilgi yok [1],[2],[3]" diyor. O atıflar hiçbir iddiayı desteklemiyor; arayüzde yanıltıcı kaynak listesi çıkar.
OpenAI-uyumlu adaptör	Tek adaptörle OpenAI/Groq/Together/vLLM/LM Studio + Ollama'nın /v1 ucu. "Dizüstünde küçük model, sunucuda büyük model" bir yapılandırma kararına iner.
Anahtar değil, anahtarın ADI tutulur	Config loglansa bile sır sızmaz.
Preflight + LLM ısıtma	Sorunu ilk soruda 500 ile değil, açılışta ve eyleme dönüştürülebilir mesajla bildirir. Ayrıca modeli belleğe rezerve eder (sıra kritik).
Bağlam bütçesi = pencerenin yarısı	Kalanı talimat + cevap için. Aşarsa model metnin başını/sonunu görmez ve bunu söylemez.

④ Hangi hatayı imkânsız kılıyor

- Boş bağlamla model çağırarak
- Modelin "bilmiyorum" diyememesi
- Uydurulmuş atfın gerçek sayılması
- Reasoning izinin atfı ayrıştırmasını kirlenmesi
- Tek sağlayıcıya kilitlenmek
- Prompt değişikliğinin izlenemez olması

KANIT — HALÜSİNASYON KONTROLÜ ÇALIŞIYOR

SORU: "Titanik gemisi hangi yıl battı?"
(model cevabı KESİNLİKLE biliyor – 1912)
CEVAP: "Bu bilgi verilen dokümanlarda yok" → çekimser=True, atıf=0

5 negatif sorunun 5'inde de çekimser kaldı. Dünya bilgisi sızmadı.

SORU: "SmartSafe AI kaç PPE sınıfı tespit ediyor?"
CEVAP: "SmartSafe AI, 17 PPE sınıfını gerçek zamanlı tespit eder [1]."
→ [1] SmartSafe AI.pdf · s.1 (doğru kaynak)

MÜLAKAT

1. Halüsinasyonu azaltmak için promptta hangi üç şeyi mutlaka yazarsın?
2. Prompt'u neden dosyada versiyonlu tutuyorsun?
3. Düşünen (reasoning) bir model kullanırken atfı ayrıştırmasında ne riski var?
4. Tek sağlayıcıya bağlı kalmamak için ne yaptın?

Evaluation Harness

evaluation/ (golden_set · metrics · runner · report)

① Bu nedir

Sistemin kalitesini **ölçen** altyapı. Golden set + metrikler + teşhis raporu.

Cevapladığı soru: "İyileştirdim mi, yoksa öyle mi sanıyorum?"

② Nasıl çalışır

Golden set (17 soru: 12 cevaplanabilir + 5 KASITLI cevaplanamaz)

- her soruyu sisteme sor
- İKİ AYRI grup metrik hesapla
- başarısızlıkları sebepleriyle listele
- koşu ayarlarını kaydet, zaman damgalı dosyaya yaz

③ Hangi karar neden

En kritik karar: doğru cevap NEYE bağlanır?

Naif: beklenen `chunk_id` 'leri kaydetmek. **Neden yanlış:** chunk kimliği içerikten türüyor. Chunk boyutunu değiştirdiğin an tüm kimlikler değişir ve golden set çöp olur — yani **ölçmek istediğin şeyi değiştirdiğinde ölçü aletini kaybediyorsun.**

Doğru: (`dosya`, `sayfa`) çiftine bağla. Sayfa numarası chunking'den bağımsızdır; dokümanın kendi gerçeğidir.

Negatif örnekler zorunlu

Yalnızca cevaplanabilir sorularla ölçmek, sistemin **en tehlikeli hatasını** (uydurma) hiç görmemektir. Golden set'te bilinçli olarak dokümanda **olmayan** sorular bulunur. "Uydurma oranı" ancak böyle ölçülebilir. Set kendini doğrular: negatif örnek yoksa uyarır.

Karar	Gerekçe
Retrieval ve generation ayrı ölçülür	Tek "doğruluk" sayısı hangi katmanın bozuk olduğunu gizler. Retrieval bozukken prompt iyileştirmek boşa emektir — model elinde olmayan bilgiyi üretemez.
Tam metin değil, kritik olgu kontrolü	LLM aynı bilgiyi farklı cümlelerle ifade eder; bu hata değildir. Sayı/oran/isim kontrolü hem sağlam hem anlamlı.
Retrieval, used_chunks üzerinden	Modele GERÇEKTEN giden bağlam ölçülür. Ayrıca retrieve etmek iki kat yavaş olur ve farklı sonuç ölçme riski doğar.
Koşu ayarları kaydedilir	"Recall@5 = 0.82" tek başına hiçbir şey ifade etmez. Hangi embedder? Hangi chunk boyutu? Reranker açık mıydı? Kaydedilmezse iki koşu karşılaştırılmaz.
Sonuçlar zaman damgalı , üst üste yazılmaz	Değerlendirmenin değeri tek sayıda değil, sayının zaman içindeki hareketinde .
Rapor teşhis üretir	Metriklerin birlikte okunuşu: Hit yüksek + MRR düşük = reranker problemi. Bu yorumu rapor yapmalı, kullanıcı ezberlemek zorunda kalmamalı.
--fail-under	CI kapısı: kalite gerilemesi derlemeyi kırar.

④ Hangi hatayı imkânsız kılıyor

- "Gözle bakıyorum, iyi görünüyor" ile karar vermek
- Chunking değişince golden set'i kaybetmek
- Uydurma oranını hiç ölçmemek
- Hangi katmanın bozuk olduğunu bilmeden emek harcamak
- İki koşuyu yanlışlıkla karşılaştırmak
- Gerilemeyi fark etmemek

KANIT — HARNESS BİR GERİLEMİYİ YAKALADI

Prompt v2 hipotezi: "niteleyiciyi birebir eşleştir" talimatı eklenirse zaman aralığı karışması düzelir.

Sonuç: HİPOTEZ YANLIŞ.

	v1	v2	
Geçme oranı	%94,1	→ %88,2	▼
Çekimserlik doğr.	%100	→ %94,1	▼
Uydurma oranı	%0	→ %5,9	▼▼

Hedeflenen hata DÜZELMEDİ, üstüne YENİ bir uydurma çıktı.
DERS: prompt'a kural eklemek mevcut kuralları ZAYIFLATIR.

Golden set olmasa v2 "iyileştirme" diye yayınlanmıştı.

MÜLAKAT

1. RAG sisteminin iyileştiğini nasıl kanıtlarsın?
2. Retrieval ve generation hatalarını nasıl ayırt edersin? Hangisini önce ölçersin?
3. Golden set'te beklenen sonucu neye bağlarsın, neden chunk kimliğine değil?
4. Negatif örnek olmadan neyi ölçemezsin?

Kompozisyon Kökü · API · Observability · CLI

composition.py · api/ · observability.py · cli.py

Kompozisyon kökü — composition.py

Tüm gerçek bileşenler **tek yerde** kurulur ve birbirine bağlanır. Başka hiçbir modül "hangi implementasyonu kullanacağım?" sorusunu sormaz.

- **API, CLI ve evaluation aynı fabrikayı kullanır** → "CLI'da çalışıyor ama API'de farklı" sınıfı hatalar yapısal olarak imkânsız.
- Bağımlılık grafiği tek dosyadan okunabilir.

Yükleme sırası tesadüf değil, kodun sözleşmesi

1. LLM ısıtılır → Ollama modeli belleğe/VRAM'e alır, keep_alive ile tutar
2. Embedder (2,3 GB)
3. Reranker (tembel)

Ters sırada: "model requires more system memory (3.8 GiB)
than is available (1.1 GiB)" ← ölçüldü

API katmanı — api/app.py · api/schemas.py

Karar	Gerekçe
Uç noktalar <code>def</code> , <code>async def</code> DEĞİL	En sık yapılan FastAPI hatası: <code>async def</code> içinde bloklayan iş (model çıkarımı) olay döngüsünü kilitler ve tüm sunucu tek soru boyunca durur — sağlık kontrolü bile cevap vermez. Senkron tanımlanınca FastAPI iş parçacığı havuzunda çalıştırır.
HTTP şeması domain'den ayrı	İç yapı özgürce değişir; chunk'ın tam metni sızmaz (sadece önizleme).
<code>lifespan</code> ile tek seferlik yükleme	Yoksa her soru 15+ sn model yükleme bekler.
liveness ≠ readiness	<code>/health</code> süreç ayakta mı · <code>/ready</code> trafik alabilir mi. Karıştırmak, LLM geçici düştüğünde sağlıklı sürecin yeniden başlatılmasına yol açar.
Boş index → 409 , 500 değil	Önkoşul eksikliği sunucu hatası değildir.
<code>top_k</code> istek başına parametre	Paylaşılan nesnenin alanını değiştirmek eşzamanlı isteklerde yarış koşulu olurdu.
Yığın izi istemciye gitmez	Dosya yolları ve iç yapı sızdırır. Loga gider; istemciye <code>request_id</code> verilir.

Observability — observability.py

`print` değil, **yapılandırılmış olay**:

```
retrieval.completed query=... per_retriever={dense:6,sparse:6}
fused=6 consensus=6 returned=3 ms=1360
```

Alan adıyla filtrelenebilir, JSON'a çevrilip log toplayıcıya verilebilir. Her isteğe `request_id` → o isteğin tüm satırları bir arada görülebilir. Kütüphane kodu **asla** stdout'a yazmaz.

```
rag-ingest [--force] · rag-ask "..." [--stream --json --show-context]  
rag-eval [--k N] [--fail-under 0.85]
```

- Anlamalı çıkış kodları (0/1/2) → CI ve betikler bunlara güvenir
- `--json` → insan çıktısı ile makine çıktısı karıştırılmaz
- `rag-ingest` LLM'i ısıtmaz → 3,5 GB boşa ayrılmaz

Bulunan Sessiz Hatalar

Hepsinin ortak özelliği: hiçbir hata mesajı vermiyordu. Sistem "çalışıyor" görünüyordu. Hepsini yakalayan şey **ölçüm ve test**, kod okumak değil.

#	Hata	Nasıl bulundu / etkisi
1	Chunk kimliği <code>id()</code> (bellek adresi)	RRF fusion hiç çalışmıyordu ; uzlaşma her zaman 0
2	Chunk'lar 256 token limitini aşıyor	8 chunk'ın 4'ü kesik, 420 token görünmez
3	Vektör store iki kez yükleniyor	Nesne kimlikleri farklı → birleştirme imkânsız
4	Index dosya adı uyuşmazlığı	Sistem hiç açılmıyordu (sonsuz dek "doküman bekleniyor")
5	MiniLM Türkçe diyakritikleri siliyor	<code>cos("poliçe", "police") = 1.0000</code>
6	Örtüşme paragraf düzeyinde	Gerçek ayarlarda örtüşme fiilen kapalı
7	Tire birleştirme rakamları yakalıyor	<code>2020-\n2021</code> → <code>20202021</code>
8	Stemmer tek geçişli → asimetrik gövde	<code>poliçe</code> ≠ <code>poliçenin</code> ; stemmer amacının tersini yapıyor
9	Sonek listesi uzunluğa göre sıralı değil	<code>gövdesi</code> → <code>gövdes</code> ("si" hiç denenmedi)
10	Ünsüz yumuşaması kaçırılıyor	<code>güvenlik</code> ≠ <code>güvenliğin</code>
11	BM25 elemesi <code>score > 0</code>	IDF=0 olan gerçek eşleşmeler atılıyor; sparse 2 → 4
12	Türkçe İ tuzağı <code>answeren</code> 'da	Büyük harfli çekimser cevap "uydurma" sayılıyordu
13	Ollama CUDA backend'i olmadan kurulu	4 → 60,5 token/sn (15 kat)
14	Reranker OS tarafından öldürülüyor	Python istisnası YOK → <code>try/except</code> yakalayamaz
15	Her açılıшта HuggingFace Hub'a sorulması	Hub yavaşlayınca tamamen yerel servis düştü
16	Reasoning izi (<code><think></code>) ayıklanmıyor	İz içindeki <code>[1]</code> gerçek atıf sanılır
17	Atıflar yanlış sayfayı gösteriyor	Yeni metrik eklendi; atıf doğruluğu %66,7 çıktı

Deney günlüğü — ölçülmüş üç karar

Deney	Hipotez	Sonuç
1 · prompt v1	Temel	geçme %94,1 · uydurma %0 · atıf doğr. %66,7
2 · prompt v2	"Niteleyiciyi birebir eşleştir" talimatı hatayı düzeltir	BAŞARISIZ. Hata düzelmedi, üstüne yeni uydurma çıktı (%0 → %5,9). Ders: kural eklemek mevcut kuralları zayıflatır.
3 · <code>top_k</code> 5→3	Küçük model 5 kaynak numarasını takip edemiyor	BENİMSENDİ. atıf doğr. %66,7→%91,7 · olgu doğr. %91,7→%100 · geçme %70,6→%88,2 · Recall %100 (kayıp yok). Ödünleşim: uydurma %0→%5,9.

NİHAİ DURUM

Hit@k / Recall@k	%100 / %100
MRR	%83,3
Çekimserlik doğruluğu	%94,1
Uydurma oranı	%5,9
Atıf oranı / doğruluğu	%100 / %91,7
Olgu doğruluğu	%100
GEÇME ORANI	%88,2
143 test · ~2 saniye · model yüklemeyen	

Üç transfer edilebilir ders

1. **"Daha çok bağlam daha iyi cevap" yanlış.** Fazla bağlam modeli kaynak eşlemede yanıltıyor. Prompt'la çözülemeyen problem, bağlamı daraltarak çözüldü — sorun talimat değil **dikkat dağınıklığıydı**.
2. **Prompt'a kural eklemek her kuralı zayıflatır.** Küçük modellerde bu etki belirgin. Çekimserlik kuralı en kritikse, onun baskınlığını azaltan hiçbir değişiklik kabul edilemez.
3. **Ölçü aletinin kendisi de test edilmeli.** Kendi harness'ımdaki boşluğu (atıf doğruluğu) onu kullanırken buldum. Metrik bozursa sistem olduğundan iyi/kötü görünür.